GlossX: An Open, Plugin-Native Agent Harness for Research and Learning

Cai Jiechao* UID: 3036791526
Cai Zhuorui UID: 3036xxxxxx
Sun Zhiwei UID: 3036xxxxxx
Zhang Guanyi UID: 3036793811

The University of Hong Kong

1 Project Objective

What Gloss is today. Gloss [13] is our open-source, self-hosted AI reading companion, a local re-implementation of the Moonlight [17] idea (v1.0.2, July 2026), available as a Windows desktop app and as an npm package for macOS and Linux. A FastAPI backend serves a React reader built on PDF.js [18] and PyMuPDF [6], and drives four interchangeable providers: the local Claude Code [5] and Codex [20] CLIs, the Anthropic API, and any OpenAI-compatible endpoint (e.g., HKU's GenAI service or vLLM). Its features are grounded in the paper: structured summaries; deep reading notes in the style of the DeepPaperNote skill [1]; an interactive React Flow [31] mind map inspired by Understand-Anything [12] (∼85k GitHub stars); cached sentence-level translation; BM25-grounded chat; PDF-anchored auto-highlights; Crossref/arXiv reference resolution; and Semantic Scholar/arXiv search. It also ships a declarative plugin marketplace with nine plugins, a structure similar to DSH's "everything is a plugin" design [10] but released earlier: Gloss v1.0.1 shipped it on 21 July 2026, three weeks before DSH appeared on 13 August.

The gap. Gloss is not yet an agent. Every feature is a single-prompt pipeline and the *user* is the planner. Plugins are prompt templates that cannot call tools or each other. The local CLIs run with all tools disabled, so their coding ability is unused.

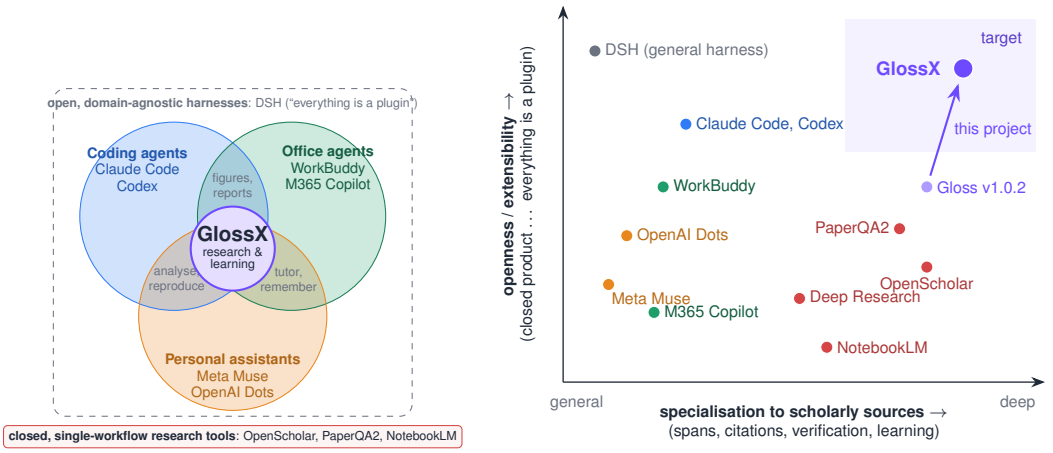
Objective. Build GlossX, an open agent harness specialised for research and learning in which *everything is a plugin* and a core agent decides what to do next. Measurable goals: **(O1)** a small kernel and one plugin ABI, so that new workflows need *zero* kernel changes; **(O2)** agentic workflows (literature review, cross-paper QA, claim verification, reproduction, tutoring) that outperform Gloss's own single-prompt pipelines while every claim is traceable to a source span; **(O3)** a systematic evaluation of quality, reliability, latency, cost and extensibility.

2 Motivation and Challenges

Students and researchers switch between a PDF reader, a reference manager, a chatbot, a coding agent and an office suite, and they glue the results together by hand. General agents produce fluent but unverifiable text: on ScholarQABench, GPT-4o fabricates 78–90% of its citations [7], and a recent survey names verification as the main gap of AI-scientist systems [11]. The tasks are also hard: the best agentic workflow scores only 38.8% on cross-paper PaperArena questions [29]. Figure 1 shows the gap in the market. Coding agents master code but have no notion of sources or evidence. Office agents [26] and assistants [16, 21] act on apps and errands rather than scholarly corpora. Research tools (OpenScholar [7], PaperQA2 [23], NotebookLM [14]) are closed, single-workflow systems. Open harnesses such as DSH [10] are domain-agnostic. GlossX targets the intersection: a research-and-learning harness that is open like DSH and *delegates* code to existing coding agents instead of re-implementing them.

*Contact member: u3679152@connect.hku.hk

Course project proposal (Agentic AI), October 2026.



(a) Intersection view (b) Two-dimensional view (qualitative)

Figure 1: **Positioning.** (a) Research needs code (analyse, reproduce), office output (reports, tables) and assistant behaviour (tutoring, memory); GlossX sits at their intersection. (b) Coding, office and assistant agents are extensible but not specialised to scholarly sources; research tools are specialised but closed, single-workflow systems; DSH is open but domain-agnostic. GlossX targets the open *and* specialised corner, starting from Gloss v1.0.2. Placements are our qualitative assessment.

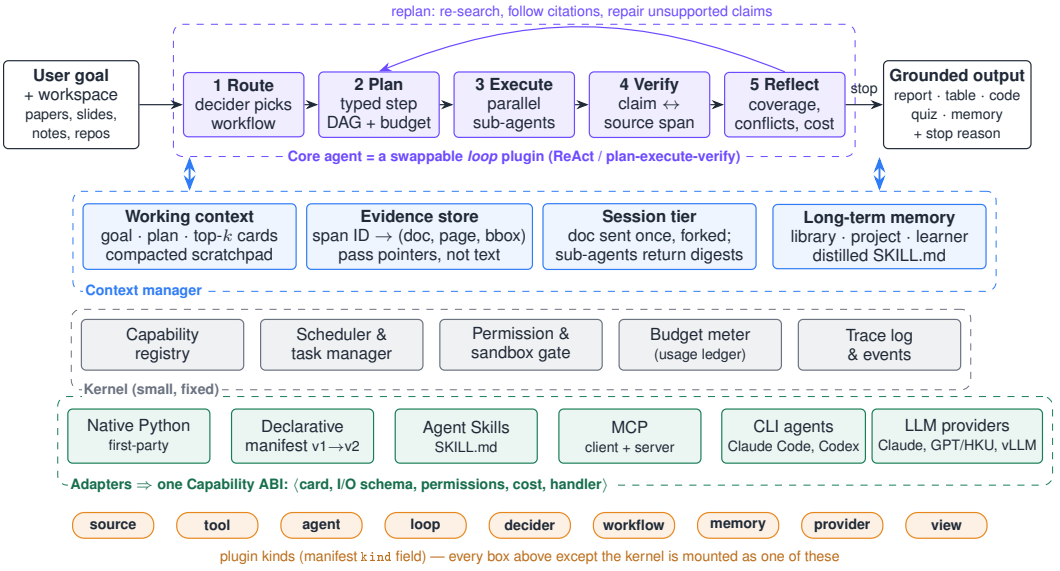


Figure 2: **GlossX architecture.** A fixed kernel; everything else, including the core loop, is a plugin compiled by an adapter into one Capability ABI. The core agent routes, plans, executes, verifies and reflects, and the context manager keeps full texts out of the planner's window.

Challenges. C1 Unification: Python functions, prompt plugins, SKILL.md skills, MCP servers, CLI coding agents and specialised models must look the same to a single planner. **C2 Context:** one paper is 10–30k tokens; multi-paper workflows overflow the context window and multiply cost when text is re-sent. **C3 Grounding:** outputs must be verifiable at span level, not merely "cited". **C4 Openness vs. safety:** third-party plugins, code execution, and documents that contain prompt injections. **C5 Evaluation:** outputs are open-ended and the API budget is a student budget.

Table 1: **Plugin roadmap.** Top: existing Gloss modules and how v2 upgrades them. Bottom: new plugins. Tier: **C** = attempted first, S = stretch. This is a candidate list; the final set depends on prototyping and progress. Dataset details are in App. C.

Plugin (*kind*)	**What it does in v2**	**Validation**	**Tier**
Existing modules → agent capabilities			
Summary, Deep Note, Mind Map (*agent*)	emit span-anchored claims that must pass the verifier before saving	span support rate; QASPER [9] evidence recall vs. B0	C
Chat (*workflow*)	BM25 RAG → agentic QA that picks retrieval, tables, figures, other papers or web; multi-hop	PaperArena [29] acc.; QASPER F1	C
Scholar, References (*tool*)	single API calls → search tools with fallback chains (arXiv → S2 → Zotero MCP → library), driven by the review loop	LitSearch [2] Recall@k; AutoResearchBench [30]	C
Evidence Table, Critical Review (*agent*)	every row and critique is checked against spans; conflicts across papers are flagged	ArxivDIGESTables [19]; ScholarQABench citation P/R	C
Repro. Checklist, Impl. Blueprint (*workflow*)	checklist → plan → code and run via Claude Code/Codex in a sandbox → compare with reported numbers	PaperBench Code-Dev [25] rubric (3–5 papers)	C
Reading Plan, Glossary, Equation/Figure Guide, Slides Outline (*workflow*)	one study-and-tutor workflow over papers, slides and notes: concept map → quiz → adapt to a learner model	TutorBench [24] subset; course slides, human rating	C
Translate, Explain, Highlight, Skills (*tool*)	become tools other agents call; highlights become evidence spans; SKILL.md skills load by progressive disclosure	anchoring precision; routing accuracy with/without skills	C
New plugins			
Agent cluster (*agent+workflow*)	planner, N parallel readers, critic and writer; map–reduce plus critic-vs-author debate over spans	single vs. $N{=}4/8$ swarm: accuracy, latency, cost on PaperArena/AutoResearchBench	C
Loop strategies (*loop*)	ReAct, plan–execute–verify and Reflexion-style loops [22] with explicit stop reasons; scheduled *Research Radar*	quality vs. budget over 1/2/3 rounds; non-termination rate; radar precision@10	C
Verification (*agent+decider*)	atomic claims → span retrieval → entailment; citation existence via Crossref/arXiv; cross-source consistency	SciFact [28] acc./F1, ECE; fabricated-citation rate	C
MCP bridge, new sources (*tool/source*)	Zotero/arXiv MCP tools in, library and spans out; Markdown, slides and web pages into the span store	interop with Claude Code/Codex; QA parity across formats	C
Jev decider (*decider*)	typed, calibrated decisions for routing, screening, verification and stopping [27]; LLM+JSON fallback	accuracy, ECE, latency, cost vs. LLM decider on Route-100 and SciFact	S
Self-evolution (*memory*)	distil successful traces into tested SKILL.md files and retrieve them next time	held-out re-runs: success, tokens, steps; regression rate	S
A2A gateway (*provider*)	expose GlossX workflows as A2A agents [15]; delegate to external agents (e.g., office agents for slides)	hand-off success and latency in interop tests	S

3 Potential Solutions: the GlossX Approach and Methodology

3.1 Everything is a plugin: one Capability ABI (C1)

The kernel is deliberately small: a capability registry, a scheduler (Gloss's existing task manager, which already supports unbounded concurrency, cancellation and progress), a permission and sandbox gate, a budget meter (the existing usage ledger), and a trace log (Fig. 2). Everything else is a plugin with a manifest (`api_version: 2`, `kind` ∈ {source, tool, agent, loop, decider, workflow, memory, provider, view}; example in App. A). Adapters compile every plugin into a *Capability* ⟨card, I/O schema, requirements, permissions, cost and side-effect hints, handler⟩. The *card* is a description of at most 100 tokens that the planner sees; the full schema and instructions are loaded only when the capability is selected (progressive disclosure, as in Agent Skills [4]). There are five adapters: (i) *native Python* for trusted first-party plugins; (ii) *declarative manifests*, so all nine v1 plugins upgrade automatically into tools whose prompt becomes the handler; (iii) *SKILL.md* skills; (iv) *MCP* [3], where GlossX is both a client (e.g., Zotero and arXiv servers become tools) and a server (exposing the library, spans and annotations to external agents); (v) *CLI agents*, where Claude Code and Codex become sandboxed `delegate` tools with their tools enabled in a per-project workspace. Because the loop and the decider are plugins too, a ReAct loop [32], a plan–execute–verify loop, or a Jev-backed decider [27] can be swapped in without touching the kernel.

3.2 How the core agent invokes plugins

(1) Route. A decider maps the goal and workspace to a workflow plugin, or to the generic loop, using capability cards (BM25 retrieves the top-k cards when the registry is large). **(2) Plan.** A planner emits a typed step graph (DAG). Each step names a capability, its inputs (span IDs or artifacts), a success criterion and a budget. **(3) Execute.** The loop dispatches ready steps, and independent steps (e.g., one reader per paper) run in parallel as kernel tasks. Every call passes the permission gate

and budget meter and is traced. **(4) Verify.** A verifier splits outputs into atomic claims and retrieves candidate spans; a decider labels each claim supported, partial or unsupported with a confidence. Unsupported claims are repaired or removed. **(5) Reflect.** Using sub-question coverage, conflicts and remaining budget, the loop replans (re-search, follow citations, read further) or stops with an explicit reason: goal met, budget exhausted, no progress for k rounds, or approval needed [22]. Tool failures, timeouts and offline sources become observations, so the agent retries with an alternative capability (e.g., arXiv $\rightarrow$ Semantic Scholar $\rightarrow$ local library).

3.3 Context management: pass pointers, not text (C2, C3)

Working context holds only the goal, the plan, the current step, the top-k cards and a scratchpad. Older steps are compacted into summaries that keep their IDs. **Evidence store:** source plugins normalise PDFs, LaTeX, Markdown, slides and web pages into *spans* with stable IDs $\rightarrow$ (doc, page, bbox). Agents exchange span IDs, and text is materialised under a token budget only when it is needed. The same IDs make every final claim clickable back to its location in the reader. **Session tier:** Gloss's shared paper session sends a document once per provider session, and sub-agents fork it. Each reader works in an isolated context with only its document and returns a schema-bound digest (map–reduce), so the orchestrator never holds full texts. **Long-term memory** covers the library, project memory (findings, decisions, preferences), a learner model, and SKILL.md files distilled from successful traces for reuse in later tasks.

3.4 Openness vs. safety (C4)

First-party plugins may contain code; third-party plugins stay declarative compositions of existing capabilities, keeping Gloss's rule of never executing downloaded code. Code runs only in sandboxed delegates under approval and budget, MCP servers get scoped permissions, and document text and tool outputs are untrusted data that can never issue tool calls.

3.5 Plugin roadmap: upgrading existing modules and adding new ones

Gloss already ships ten built-in features and nine marketplace plugins (six more are listed as planned), but each is a one-shot prompt. Table 1 shows how they become v2 capabilities that agents can call, and which new plugins we add; each row gives the change and how it will be validated. The list is a candidate set, not a commitment: which plugins we finally implement, and in what order, will be decided by early prototyping and our progress. Together they cover every agentic capability in the project brief.

4 Experimental Dataset Description and Evaluation Plan

Data sources. (a) The public benchmarks in Table 1, subsampled to 50–100 items each to fit the API budget. (b) *GlossX-Route-100*: 100 heterogeneous research and study requests (find, compare, verify, reproduce, learn) labelled with their gold workflow and capabilities and cross-checked by group members. (c) Slides and notes from this course. (d) A fault-injection suite: offline network, provider timeouts, malformed PDFs, failing plugins, and instructions injected into documents.

Metrics. *Quality:* each benchmark's native metric plus citation precision/recall and the span-level support rate. *Reliability:* task success, recovery rate under faults, and variance over 3 runs. *Efficiency:* latency, tokens, cost (from the usage ledger) and tool calls. *Extensibility:* lines needed to add a workflow, and the kernel diff (target: none).

Baselines and protocol. B0 is Gloss v1.0.2's single-prompt features, B1 is standard RAG, and B2 is GlossX. Besides the per-plugin comparisons in Table 1, ablations remove the verifier or reflection, compare full context with pointers plus a shared session, and swap model backends (HKU GPT models, Claude, Codex, an open model on vLLM). An LLM judge scores rubric tasks and humans spot-check 20% (Cohen's κ). Failures are categorised as retrieval miss, wrong tool, unsupported claim, non-termination or budget overrun.

Success criteria. B2 beats B0 and B1 on PaperArena accuracy and citation precision at comparable cost; routing accuracy on Route-100 is at least 90%; the last three workflows need no kernel changes; and every fault scenario degrades gracefully.

References

[1] 917Dhj. DeepPaperNote: An agent skill for deep-reading a single paper. https://github.com/917Dhj/DeepPaperNote, 2026. MIT license.

[2] Anirudh Ajith, Mengzhou Xia, Alexis Chevalier, et al. LitSearch: A retrieval benchmark for scientific literature search. In *Proceedings of EMNLP*, 2024.

[3] Anthropic. Model context protocol. https://modelcontextprotocol.io, 2024.

[4] Anthropic. Agent skills: An open standard for portable agent capabilities (SKILL.md). https://agentskills.io, 2025.

[5] Anthropic. Claude code. https://claude.com/claude-code, 2025.

[6] Artifex. PyMuPDF: Data extraction and analysis for PDF documents. https://github.com/pymupdf/PyMuPDF, 2012.

[7] Akari Asai et al. OpenScholar: Synthesizing scientific literature with retrieval-augmented LMs. *arXiv preprint arXiv:2411.14199*, 2024.

[8] Jonathan Bragg, Mike D'Arcy, Nishant Balepur, et al. AstaBench: Rigorous benchmarking of AI agents with a scientific research suite. *arXiv preprint arXiv:2510.21652*, 2025. ICLR 2026.

[9] Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A. Smith, and Matt Gardner. A dataset of information-seeking questions and answers anchored in research papers. In *Proceedings of NAACL*, 2021.

[10] DeepSeek-AI. DeepSeek Harness (dsh): An open-source agent harness where everything is a plugin. https://www.orcarouter.ai/blog/deepseek-harness-explained, 2026. Released 13 August 2026, MIT license.

[11] Tianyu Ding, Aditya Nannapaneni, Bingfan Liu, et al. Autonomous research agents: A survey of AI scientists and the verification gap. *arXiv preprint arXiv:2608.05179*, 2026.

[12] Egonex-AI. Understand-anything: Turn any code into an interactive knowledge graph. https://github.com/Egonex-AI/Understand-Anything, 2026. About 85k GitHub stars as of October 2026, MIT license.

[13] Gloss contributors. Gloss: A local, self-hosted AI companion for reading research papers (v1.0.2). https://github.com/computersniper/gloss, 2026.

[14] Google. NotebookLM. https://notebooklm.google, 2024.

[15] Google and the Linux Foundation. Agent2agent (A2A) protocol. https://a2a-protocol.org, 2025.

[16] Meta. Meta launches Muse, a personal AI agent. https://www.techrepublic.com/article/news-meta-muse-ai-agent-us-launch/, 2026. Announced 8 September 2026.

[17] Moonlight. Moonlight: Your AI colleague for reading papers. https://www.themoonlight.io, 2025.

[18] Mozilla. PDF.js: A PDF reader in JavaScript. https://github.com/mozilla/pdf.js, 2011.

[19] Benjamin Newman et al. ArxivDIGESTables: Synthesizing scientific literature into tables using language models. In *Proceedings of EMNLP*, 2024.

[20] OpenAI. Codex CLI. https://github.com/openai/codex, 2025.

[21] OpenAI. OpenAI debuts dots, always-on assistant agents. https://axios.com/2026/09/29/openai-dots-ai-assistant-devday, 2026. DevDay, 29 September 2026.

[22] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.

[23] Michael D. Skarlinski et al. Language agents achieve superhuman synthesis of scientific knowledge. *arXiv preprint arXiv:2409.13740*, 2024.

[24] Rakshith S. Srinivasa, Zora Che, Chen Bo Calvin Zhang, et al. TutorBench: A benchmark to assess tutoring capabilities of large language models. *arXiv preprint arXiv:2510.02663*, 2025.

[25] Giulio Starace et al. PaperBench: Evaluating AI's ability to replicate AI research. *arXiv preprint arXiv:2504.01848*, 2025.

[26] Tencent. Tencent launches WorkBuddy productivity AI agent. https://technode.com/2026/05/29/tencent-launches-workbuddy-productivity-ai-agent-for-global-users/, 2026.

[27] TypeSafe AI. Jev: A system one model returning typed, calibrated decisions. https://www.datacamp.com/blog/system-one-models-jev, 2026. Announced 15 September 2026.

[28] David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, Arman Cohan, and Hannaneh Hajishirzi. Fact or fiction: Verifying scientific claims. In *Proceedings of EMNLP*, 2020.

[29] Daoyu Wang, Mingyue Cheng, Shuo Yu, et al. PaperArena: An evaluation benchmark for tool-augmented agentic reasoning on scientific literature. *arXiv preprint arXiv:2510.10909*, 2025.

[30] Lei Xiong, Kun Luo, Ziyi Xia, et al. AutoResearchBench: Benchmarking AI agents on complex scientific literature discovery. *arXiv preprint arXiv:2604.25256*, 2026.

[31] xyflow. React Flow: Libraries for building node-based UIs. https://github.com/xyflow/xyflow, 2019.

[32] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.

A Example plugin manifest (v2)

Version-1 manifests remain valid; `kind`, `capability` and `steps` are the v2 additions. A workflow is a declarative composition of other capabilities, so third-party authors never ship code.

```
{
  "api_version": 2,
  "id": "glossx.literature-review",
  "kind": "workflow",
  "name": "Literature Review",
  "permissions": ["library:read", "search:web", "ai:complete"],
  "capability": {
    "card": "Answer a research question with a cited review: plan sub-questions, search, read papers in
         parallel, verify every claim, and re-search until coverage is met.",
    "input": {"question": "string", "max_papers": "integer"},
    "output": {"report": "markdown", "claims": "claim[] with span_ids"}
  },
  "steps": [
    {"id": "plan", "use": "agent:planner"},
    {"id": "search", "use": "tool:scholar.search", "fallback": ["mcp:zotero.search", "tool:library.search
         "]},
    {"id": "screen", "use": "decider:relevance"},
    {"id": "read", "use": "agent:reader", "map_over": "screen.accepted", "context": "shared_session"},
    {"id": "verify", "use": "agent:verifier"},
    {"id": "write", "use": "agent:writer"}
  ],
  "loop": {"use": "loop:plan-execute-verify", "stop": {"coverage": 0.8, "max_rounds": 3, "max_usd": 0.5}},
  "agent": {"name": "Review Scout", "icon": "telescope"}
}
```

B Workflow example: literature review

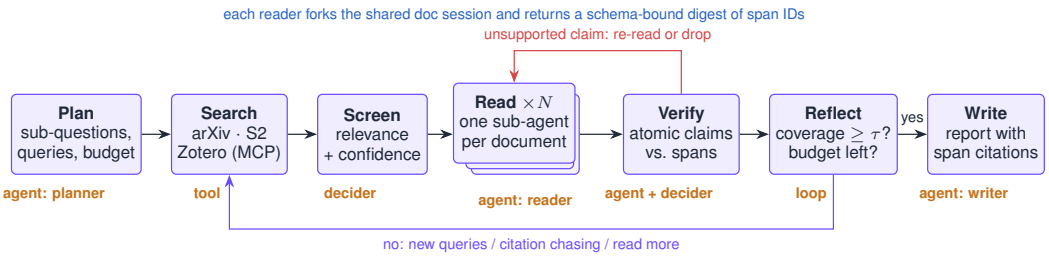


Figure 3: The literature-review workflow plugin. Each step is a capability of the indicated kind. The reflect step loops back to search when coverage is below τ, and the verifier sends unsupported claims back to the readers.

C Benchmark details

Benchmark	Content	Why it fits GlossX	Planned n
PaperArena [29]	784 tool-augmented cross-paper QA pairs (AI papers)	multi-step, multi-paper tool use; best published workflow 38.8%	100
AutoResearchBench [30]	expert-verified literature discovery; Deep and Wide research tasks	tests the search–reflect loop and coverage stopping	50
LitSearch [2]	597 literature-search queries (ML/NLP)	cheap, stable retrieval recall	100
ScholarQABench [7]	2,967 expert queries, 208 long-form answers	citation precision/recall of synthesised answers	50
QASPER [9]	QA over NLP papers with evidence annotations	span grounding of single-paper answers	100
SciFact [28]	scientific claim verification	verifier and decider accuracy and calibration	100
PaperBench Code-Dev [25]	rubric-graded code development for ICML papers, no GPU needed	delegation to coding agents	3–5
TutorBench [24]	1,490 rubric-graded tutoring samples	explanation, feedback and hint quality	60
ArxivDIGESTables [19]	literature-review tables (AstaBench [8] subset)	multi-paper aspect extraction	30
GlossX-Route-100 (ours)	100 heterogeneous requests with gold workflow/capabilities	routing and tool selection across plugins	100

D Timeline and division of labour

Weeks (2026)	Milestone	Owner(s)
9 Oct	proposal submitted	all
10 Oct – 19 Oct	kernel, Capability ABI, v1→v2 adapter, trace log	Cai Jiechao (lead), Cai Zhuorui
20 Oct – 2 Nov	core loop, router/planner/verifier, context manager; MCP/Skills/CLI adapters	Cai Jiechao, Cai Zhuorui, Sun Zhiwei
3 Nov – 16 Nov	prototype candidate plugins (Table 1) and choose which to complete	Cai Zhuorui, Sun Zhiwei, Zhang Guanyi
10 Nov – 24 Nov	benchmark harness, GlossX-Route-100, fault injection, ablations	Zhang Guanyi (eval lead), all
25 Nov – 30 Nov	feature freeze, end-to-end demo and rehearsal	all
1 or 8 Dec	Demo Day (provisional; session to be announced)	all
Dec – report due (TBD)	final experiments, report and code release	all

E Risks and limitations

- **API access and budget.** Jev access is uncertain; the decider ABI falls back to a small LLM with a JSON schema. Benchmarks are subsampled, and costs are tracked per run.
- **Judge bias and contamination.** LLM judges are spot-checked by humans, and older benchmarks may appear in model training data, so we also include 2026 benchmarks and our own tasks.
- **Coding-agent cost and safety.** Reproduction runs are sandboxed, capped by budget and limited to a few papers.
- **Scope.** Table 1 is a candidate list. C rows are attempted first, and plugins are added, merged or dropped according to prototyping results and progress.